

Software Requirements Specification

KAIROS

Knowledge-Augmented Insight & Retention Optimization System

"The right insight, at the right moment."

Formal Project Title:

An Explainable AI-Driven Customer Churn Intelligence Framework for Predictive and Actionable Retention

Final Year Major Project — B.E. / B.Tech, Semester VII

Document Version 1.0 | Prepared: August 2026

Table of Contents

Table of Contents	2
1. Introduction	4
1.1 Purpose	4
1.2 Document Conventions	4
1.3 Intended Audience	4
1.4 Project Scope	4
1.5 References	4
2. Overall Description	5
2.1 Product Perspective	5
2.2 Product Functions (Summary)	5
2.3 User Classes and Characteristics	5
2.4 Operating Environment	6
2.5 Design and Implementation Constraints	6
2.6 Assumptions and Dependencies	6
3. System Features (Functional Requirements)	6
3.1 Data Ingestion and Governance (Core)	6
3.2 Churn Prediction and Explainability (Core)	7
3.3 Drift Monitoring and Model Lifecycle (Core)	7
3.4 BI Dashboards (Core)	7
3.5 Conversational Retrieval Layer (Module B)	7
3.6 REST API (Core)	8
4. External Interface Requirements	8
4.1 User Interfaces	8
4.2 Hardware Interfaces	9
4.3 Software Interfaces	9
4.4 Communication Interfaces	9
5. Non-Functional Requirements	9
5.1 Performance	9
5.2 Security	9
5.3 Reliability and Availability	9
5.4 Maintainability	10
5.5 Scalability	10
5.6 Usability	10

6. Other Requirements	10
6.1 Evaluation Requirements.....	10
6.2 Data Requirements	10
Appendix A: Glossary	10
Appendix B: Technology Stack Summary.....	11

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for Kairos, an explainable, conversational customer churn intelligence platform. The document is intended to guide the design, development, and evaluation of the system across both semesters of the final year major project, and to serve as the guide-approved scope reference for the Semester VII prototype checkpoint.

1.2 Document Conventions

This document uses “Kairos” as the working name of the system throughout. The formal academic title, “An Explainable AI-Driven Customer Churn Intelligence Framework for Predictive and Actionable Retention,” is used on the cover page and in any formal report or paper submission. Requirements are labelled with unique identifiers (e.g., FR-1.1) and assigned a priority of High, Medium, or Low. High-priority requirements form the Core layer that must be delivered by the Semester VII checkpoint; Medium-priority requirements belong to Module B, the conversational layer, and may extend into Semester VIII if time is constrained.

1.3 Intended Audience

- Project guide and evaluation panel, for scope approval and progress assessment.
- The developer (project author), as the primary build reference across both semesters.
- Any future collaborator or reviewer who needs to understand system scope and boundaries.

1.4 Project Scope

Kairos predicts which customers of a subscription or usage-based business are likely to churn, explains the prediction in plain business terms using SHAP-based interpretability, and delivers these insights through two audience-specific BI dashboards. As a differentiating extension (Module B), the system also provides a conversational natural-language interface that allows non-technical users to query the underlying data and the model's own explanations directly, without writing SQL or reading a dashboard, with explicit awareness of whether a prediction is based on a stale or drifting model.

The system is built on a governed, synthetically-augmented relational dataset (customers, subscriptions, usage_logs, support_tickets, transactions, churn_predictions, model_registry) since no real company data is available for this academic project. Kairos is not intended to be deployed against a live production business at this stage; it is a portfolio-grade, evaluation-ready capstone system.

1.5 References

- Kairos Project Documentation, Version 2.0 (internal reference document, August 2026).
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications (structural reference for this SRS).
- Literature review reference list (Kairos Project Documentation, Section 15) — 18 verified IEEE/Scopus-indexed papers across three themes: explainable churn prediction, natural-language-to-SQL interfaces, and retrieval-augmented generation over structured data.

2. Overall Description

2.1 Product Perspective

Kairos is a new, standalone system, not an extension of an existing product. It is composed of two tightly connected layers built on one governed data store:

- Core layer: a relational data model, a leakage-safe feature engineering pipeline, an interpretable churn-prediction model with SHAP explanations, drift monitoring, and two BI dashboards.
- Module B (conversational layer): a hybrid SQL and semantic retrieval engine with a query router, an LLM-based answer synthesis component, and a formal evaluation harness.

The Core layer is the primary deliverable and load-bearing story of the project. Module B is a differentiator that extends the Core layer's outputs into a conversational interface, and its scope may flex depending on time available.

2.2 Product Functions (Summary)

- Ingest and store customer, subscription, usage, support, and transaction data in a governed PostgreSQL schema.
- Engineer leakage-safe features and train an interpretable churn-prediction model.
- Generate global and per-customer SHAP explanations for every prediction.
- Monitor the trained model for data and prediction drift over time, and flag when a prediction may be unreliable.
- Present operational insights through a Power BI dashboard and executive insights through a Tableau dashboard.
- [Module B] Accept natural-language questions and route them to SQL retrieval, semantic retrieval, or a hybrid of both.
- [Module B] Retrieve stored SHAP explanations and drift metadata as part of answering “why” questions, and compose a grounded natural-language answer.
- Expose all core functionality through a secured REST API.

2.3 User Classes and Characteristics

User class	Description	Technical skill level
Operations / support agent	Uses the Power BI dashboard and, if available, the chat interface to identify at-risk customers day to day. Not expected to know SQL or ML.	Low to medium technical skill
Retention manager / analyst	Uses both dashboards to review cohorts and segments; may ask exploratory questions via chat.	Medium technical skill
Executive / leadership	Uses the Tableau dashboard for high-level trend and revenue-at-risk review; may use chat for quick summary questions.	Low technical skill
Project guide / evaluator	Reviews system design, model performance, and evaluation results.	High technical skill

2.4 Operating Environment

- Backend: Python 3.x, Flask REST API, SQLAlchemy ORM.
- Database: PostgreSQL with the pgvector extension for embedding storage.
- ML/XAI: scikit-learn, XGBoost/LightGBM, SHAP, MLflow, SciPy.
- LLM layer: provider-agnostic, configurable between OpenAI API (production) and a locally hosted Ollama model (development, to conserve API token usage).
- BI tools: Power BI Desktop/Service and Tableau Desktop/Server, both connecting to the same governed PostgreSQL instance.
- Deployment: Docker and docker-compose for containerized local/demo deployment.

2.5 Design and Implementation Constraints

- No real company data is available; all customer, usage, support, and transaction data is either a public seed dataset or realistically generated synthetic data, conditioned on churn outcome rather than fully random.
- The project must be deliverable in two phases: a working prototype by the Semester VII checkpoint (early October 2026) and a complete system by the Semester VIII final submission (end of March 2027).
- Development-time LLM calls use a locally hosted model (Ollama) to avoid consuming production API token budget; production/demo calls use the OpenAI API.
- The Core layer must remain fully functional and demonstrable even if Module B is not fully completed by the Semester VII checkpoint.

2.6 Assumptions and Dependencies

- It is assumed that a suitable public churn dataset (e.g., a telecom customer churn dataset) is available to seed realistic customer distributions.
- It is assumed that free or student-tier access to OpenAI's API and to Power BI / Tableau is sufficient for development and demonstration purposes.
- The project depends on continued guide approval of scope at each phase gate, starting with sign-off on this SRS.

3. System Features (Functional Requirements)

Functional requirements are grouped by system feature. Each requirement is uniquely identified and assigned a priority. High-priority requirements belong to the Core layer and are required for the Semester VII checkpoint. Medium-priority requirements belong to Module B and may be completed in Semester VIII if time does not allow full completion earlier.

3.1 Data Ingestion and Governance (Core)

ID	Requirement	Priority
FR-1.1	The system shall load a public seed churn dataset and generate realistic synthetic usage, support, and transaction records conditioned on churn outcome.	High

ID	Requirement	Priority
FR-1.2	The system shall store all customer, subscription, usage, support, and transaction data in a governed PostgreSQL schema.	High
FR-1.3	The system shall validate incoming data for completeness and consistency before it is used for training or serving.	High

3.2 Churn Prediction and Explainability (Core)

ID	Requirement	Priority
FR-2.1	The system shall engineer features using a fixed snapshot cutoff per customer to prevent data leakage from future information.	High
FR-2.2	The system shall train a baseline Logistic Regression model and a boosted-tree model (XGBoost or LightGBM) to predict churn probability.	High
FR-2.3	The system shall generate global and per-customer SHAP explanations for every churn prediction and store the top contributing features alongside the prediction.	High
FR-2.4	The system shall track all model training experiments, parameters, and metrics using MLflow.	High
FR-2.5	The system shall translate SHAP and cohort analysis into plain-English business insights (driver, cohort, segment, and business-impact statements).	High

3.3 Drift Monitoring and Model Lifecycle (Core)

ID	Requirement	Priority
FR-3.1	The system shall monitor incoming data and prediction distributions for drift using statistical tests (e.g., PSI, KS-test).	High
FR-3.2	The system shall flag a prediction as potentially unreliable if the underlying model is stale or drift has been detected.	High
FR-3.3	The system shall support promotion of a newly trained “challenger” model to “champion” status based on defined performance criteria.	Medium

3.4 BI Dashboards (Core)

ID	Requirement	Priority
FR-4.1	The system shall provide a Power BI operations dashboard showing at-risk customer lists, agent workload, and drill-through detail.	High
FR-4.2	The system shall provide a Tableau executive dashboard showing churn trend, cohort retention, revenue-at-risk, and aggregated SHAP drivers.	High
FR-4.3	Both dashboards shall read from the same governed PostgreSQL data store to ensure consistent figures.	High

3.5 Conversational Retrieval Layer (Module B)

ID	Requirement	Priority
FR-5.1	The system shall accept a natural-language question from the user through a chat interface.	Medium
FR-5.2	The system shall classify each question as an aggregation, lookup, semantic, or hybrid query using a query router.	Medium
FR-5.3	The system shall translate structured questions into executable SQL and execute them against the governed database.	Medium
FR-5.4	The system shall perform semantic retrieval over embedded free-text fields (e.g., support ticket text) and embedded SHAP explanation text using pgvector.	Medium
FR-5.5	The system shall retrieve the stored SHAP explanation and drift/confidence status relevant to a customer or segment when answering a “why” question.	Medium
FR-5.6	The system shall compose a final natural-language answer using only retrieved data and retrieved explanations, and shall not present unretrieved or fabricated figures as fact.	Medium
FR-5.7	The system shall display the underlying SQL query and/or retrieved context alongside each answer for auditability.	Medium
FR-5.8	The system shall include a plain-language warning in its answer when the relevant model is stale or drifting.	Medium

3.6 REST API (Core)

ID	Requirement	Priority
FR-6.1	The system shall expose a /predict endpoint returning a churn probability for a given customer.	High
FR-6.2	The system shall expose a /batch-predict endpoint for scoring multiple customers at once.	Medium
FR-6.3	The system shall expose a /customer-detail endpoint returning a customer's profile, prediction, and SHAP explanation.	High
FR-6.4	The system shall expose a /segment endpoint returning cohort/segment-level aggregated insights.	Medium
FR-6.5	The system shall expose a /model-metrics endpoint returning current model performance and drift status.	Medium
FR-6.6	The system shall expose a /chat or /query endpoint accepting natural-language questions for Module B.	Medium
FR-6.7	All API endpoints, other than health checks, shall require authenticated access via JSON Web Tokens (JWT).	Medium

4. External Interface Requirements

4.1 User Interfaces

- Power BI operations dashboard, accessed via Power BI Desktop or Power BI Service.
- Tableau executive dashboard, accessed via Tableau Desktop or Tableau Server/Public.
- A lightweight web-based chat interface for Module B, embeddable alongside either dashboard.

4.2 Hardware Interfaces

No special hardware interfaces are required. The system is designed to run on a standard development laptop or a cloud-hosted virtual machine, with no dependency on specialized hardware beyond what is needed to locally run an Ollama model during development, if used.

4.3 Software Interfaces

Interface	Purpose
PostgreSQL 15+ with pgvector	Primary data store for all structured data and vector embeddings.
OpenAI API	Production-time LLM provider for answer synthesis in Module B.
Ollama (local)	Development-time LLM provider, used to avoid consuming production API token budget while building and testing.
Power BI / Tableau	Read-only connections to the governed PostgreSQL store for dashboard rendering.
MLflow tracking server	Local or lightweight hosted instance for experiment tracking.

4.4 Communication Interfaces

All communication between the frontend interfaces (dashboards, chat) and the backend occurs over HTTPS via the Flask REST API. The API returns JSON-formatted responses, documented via an OpenAPI specification.

5. Non-Functional Requirements

5.1 Performance

- A single customer prediction request via the /predict endpoint should return a result within 2 seconds under normal load.
- A natural-language query through Module B should return an answer within 8 seconds, acknowledging that LLM response time is a dependent external factor.

5.2 Security

- All API endpoints other than health checks shall require valid JWT authentication.
- No raw customer data shall be sent to a third-party LLM provider beyond what is minimally necessary to answer a specific retrieved query; wherever feasible, only retrieved facts and summaries are passed to the LLM rather than full raw records.

5.3 Reliability and Availability

- The system shall degrade gracefully if Module B (the conversational layer) is unavailable; the Core dashboards shall remain independently functional.
- The system shall log drift-monitoring alerts so that model staleness is never silently ignored.

5.4 Maintainability

- All ML experiments shall be tracked via MLflow to allow reproducibility and rollback to a previous model version.
- The codebase shall be containerized using Docker to ensure a consistent, reproducible environment across development and demonstration.

5.5 Scalability

While Kairos is an academic capstone and not expected to serve production-scale traffic, the schema and API are designed to reasonably extend to a larger synthetic dataset (tens of thousands of customer records) without architectural redesign.

5.6 Usability

- Dashboard visuals shall use a consistent color scheme across Power BI and Tableau so that risk levels (low, medium, high) are visually intuitive to a non-technical viewer.
- Chat responses in Module B shall be written in plain business language, avoiding raw statistical jargon, unless the user explicitly asks for technical detail.

6. Other Requirements

6.1 Evaluation Requirements

- Model performance shall be evaluated using ROC-AUC, PR-AUC, calibration, and an estimated business-impact metric (expected revenue saved).
- Module B shall be evaluated using text-to-SQL execution accuracy, RAGAS-style faithfulness and relevance scores, and a new proposed explanation-faithfulness metric that checks whether a generated “why” answer matches the model's real SHAP drivers.

6.2 Data Requirements

- Synthetic data generation shall preserve realistic statistical relationships between churn outcome and behavioural features, rather than assigning churn labels randomly, so that the trained model reflects a genuine (if synthetic) predictive pattern.

Appendix A: Glossary

Term	Definition
Churn	A customer discontinuing use of a product or service.
SHAP	SHapley Additive exPlanations; a method for explaining individual predictions of a machine learning model.

Term	Definition
Drift	A change over time in the statistical properties of input data or model predictions, which can degrade model accuracy.
RAG	Retrieval-Augmented Generation; an approach where a language model's answer is grounded in retrieved data rather than generated from memory alone.
pgvector	A PostgreSQL extension that enables storage and similarity search over vector embeddings.
Explanation faithfulness	A proposed evaluation metric checking whether a natural-language explanation accurately reflects a model's real SHAP-based reasoning.

Appendix B: Technology Stack Summary

Layer	Technologies
Database	PostgreSQL + pgvector extension
ML / XAI	scikit-learn, XGBoost/LightGBM, SHAP, Optuna, MLflow, SciPy (PSI/KS-test)
RAG / LLM	sentence-transformers, LangChain or lightweight custom orchestrator, OpenAI API (production) / Ollama (development)
Backend API	Flask, SQLAlchemy, Flask-RESTX/Smorest, Flask-JWT-Extended, Pytest
BI / Dashboards	Power BI (operations), Tableau (executive)
DevOps	Docker & docker-compose, GitHub Actions (CI)
Evaluation	Custom text-to-SQL execution-accuracy harness, RAGAS, custom explanation-faithfulness scorer